Du cadrage à l'interface

TP1, TP2, TP3 : une application de sentiment, de A à Z

A. Larhlimi

Développement et déploiement d'applications intelligentes

Trois séances de 2 h · travail en binôme

Plan

L'objectif

Les trois séances

Les ressources partagées

Streamlit

La démarche, pas à pas

CI et MLOps

Plan

L'objectif

Les trois séances

Les ressources partagées

Streamlit

La démarche, pas à pas

CI et MLOps

Ce que l'on construit en trois séances

Objectif

Une application d'analyse de sentiment d'avis clients en français, complète : un modèle entraîné, testé, et une interface qu'une personne non technique peut utiliser.

Le besoin métier : une équipe relation client reçoit des milliers d'avis par jour et les traite dans l'ordre d'arrivée. On veut faire remonter les **avis négatifs** en tête de file.

Séance	Ce que le binôme produit	Contrôles	Tests
TP1	Dépôt structuré, environnement reproductible, trois fiches de cadrage	T0 à T7	38
TP2	Le modèle : entraînement, évaluation, métriques versionnées	C1 à C5	42
TP3	L'interface de démonstration Streamlit	S1 à S4	45

Un seul dépôt, `sentiment-app`, du premier jour au dernier.

Pourquoi un problème aussi simple

Ce qui est facile

Classer un avis en positif ou négatif. Un classifieur linéaire sur des sacs de mots, entraîné en quelques secondes, sans GPU, atteint 0,92 de F1.

Ce qui est difficile

Le rendre reproductible, testé, surveillé, utilisable par quelqu'un d'autre, et redéployable après un réentraînement.

Le modèle occupe 20 lignes de code sur les trois séances. Tout le reste, c'est le métier de ce module : **l'écart entre un notebook et un produit**.

Les trois compétences visées

Compétence	Vérifiée par
Choisir une approche technique et la justifier	12 tests de cadrage, lecture des fiches
Produire un modèle reproductible et l'évaluer correctement	4 tests de modèle, `metrics.json`
Exposer le modèle à un utilisateur non technique	3 tests d'interface
Travailler à deux sur un dépôt, avec tests et CI	23 tests d'environnement, 3 d'historique

Au total 45 tests, qui tournent encore à chaque push jusqu'en semaine 11.

Plan

L'objectif

Les trois séances

Les ressources partagées

Streamlit

La démarche, pas à pas

CI et MLOps

TP1 : cadrer et préparer l'environnement

Objectif

Décider quelle approche employer pour trois besoins métier, et monter un dépôt que l'enseignant peut cloner et faire tourner en trois commandes.

Partie A, 50 min

- Trois fiches de cadrage
- Grille : (a) données étiquetées, (b) vérifiabilité, (c) coût, (d) conséquence d'une erreur
- Arbre : règles, ML, RAG, génératif

Partie B, 70 min

- Environnement virtuel
- Dépôt GitHub privé, un commit par membre
- 38 tests, intégration continue verte

Aucune ligne de code métier n'est écrite. C'est la seule séance qui ne produit rien de visible, et celle sans laquelle les deux autres ne tiendraient pas.

TP2 : entraîner, évaluer, sauvegarder

Objectif

Produire un modèle reproductible, évalué correctement, sauvegardé dans un fichier que l'interface chargera. Pas un bon modèle : un modèle dont on sait pourquoi il se trompe.

Script	Ce qu'il fait	Ce qu'il produit
`src/make_demo.py`	290 avis déterministes, dont 100 mitigés	`data/demo_reviews.csv`
`src/train.py`	Découpe, entraîne un pipeline TF-IDF et régression logistique	`sentiment.joblib`, `metrics.json`
`src/evaluate.py`	Matrice de confusion, rapport, trois erreurs commentées	une lecture, pas un fichier
`src/predict.py`	La fonction `predict()` appelée par l'interface et, plus tard, par l'API	l'interface publique du modèle

Résultats de référence : F1 macro 0,879 sur le jeu de démonstration, 0,920 sur Allociné.

TP3 : l'interface de démonstration

Objectif

Rendre le modèle utilisable par quelqu'un qui n'ouvrira jamais un terminal, sans réécrire une ligne de logique de prédiction.

Deux fichiers, deux rôles

`ui/helpers.py` : les fonctions pures, testables sans Streamlit.

`ui/app.py` : l'affichage, et rien d'autre.

Trois onglets

Un avis saisi à la main.

Un fichier CSV traité en lot.

Une page « comment ça marche ».

La question de conception de la séance : **que montrer quand le modèle hésite ?** En dessous de 60 % de confiance, l'interface affiche « à vérifier » au lieu de trancher.

Plan

L'objectif

Les trois séances

Les ressources partagées

Streamlit

La démarche, pas à pas

CI et MLOps

Trois kits qui s'empilent

Archive	Contenu	Se pose sur
`sentiment-app.zip`	Le dépôt de départ : structure, scripts, 38 tests, CI, documentation	rien, c'est la base
`tp2-semaine2-kit.zip`	`src/` (quatre scripts), `tests/test_model.py`, notebook Colab	le dépôt du TP1
`tp3-streamlit-kit.zip`	`ui/` (interface et helpers), `tests/test_ui.py`, notebook Colab, compte rendu	le dépôt après TP2

Installation d'un kit, en une commande

Décompressez **à côté** du dépôt, pas dedans, puis : `python appliquer_tp2.py ../sentiment-app`. L'option `-simulation` montre ce qui serait copié sans rien écrire. Aucun kit n'écrase le travail du précédent.

Ce que contient le kit du TP1

Élément	Rôle
`README`, `MARCHE-A-SUIVRE`, `TESTS`	Porte d'entrée, pas à pas, démarche de test
`AIDE-GIT`, `DEPANNAGE`	Commandes Git et messages d'erreur
`requirements.txt`	Dépendances de tout le semestre, versions figées
`.gitignore`, `.env.example`, `pytest.ini`, `.vscode/`	Configuration, à garder telle quelle
`docs/cadrage/`	Trois fiches à remplir, un exemple corrigé, la grille, douze cas de rechange
`scripts/`	Installation, diagnostic, initialisation du dépôt, clone de vérification
`tests/`	38 tests : environnement (23), cadrage (12), historique Git (3)
`.github/workflows/ci.yml`	Intégration continue
`exemples/`, `notebooks/`	Facultatif : aperçu des semaines suivantes, variante Colab

L'entrepôt complet

Pour les étudiants

- Les trois kits, diffusés séance par séance
- `tp1-tp2-tp3.md` et sa version PDF : la démarche complète
- `ENONCE-COMPTE-RENDU.md` et son modèle à remplir
- Les notebooks Colab

Pour l'enseignant seulement

- Les trois fiches de cadrage corrigées
- Le QCM d'introduction, 40 questions
- Le guide du TP1 : figures, scripts, problèmes

Calendrier de diffusion

J-7 : prérequis (Python 3.11, Git, compte GitHub)

J-2 : le kit de la séance

Jour J : les diapositives

Après le TP3 : l'énoncé du compte rendu

Plan

L'objectif

Les trois séances

Les ressources partagées

Streamlit

La démarche, pas à pas

CI et MLOps

Streamlit en trois idées

1. Un script Python devient une page web

Pas de HTML, pas de JavaScript, pas de serveur à configurer. Chaque appel à `st.title`, `st.text_area` ou `st.dataframe` ajoute un élément à la page.

2. Chaque interaction réexécute le script de haut en bas

C'est le modèle de fonctionnement, et il surprend toujours. D'où le décorateur `@st.cache_resource` sur le chargement du modèle : sans lui, le fichier `joblib` serait relu à chaque clic.

3. Une démonstration, pas l'application

Elle sert à montrer le modèle, à récolter des retours, à faire apparaître les cas où il se trompe. C'est l'API, plus tard, qui sera appelée par d'autres programmes.

Pourquoi deux fichiers plutôt qu'un

```python
# ui/helpers.py : logique pure
def classer(texte, predicteur):
    r = predicteur(texte)
    r["incertain"] = r["score"] <
        0.60
    r["verdict"] = ("à vérifier"
        if r["incertain"] else r["
            label"])
    return r
```

```python
# ui/app.py : affichage seul
r = classer(texte, predire)
if r["incertain"]:
    st.warning("À vérifier par un
        agent")
elif r["label"] == "negative":
    st.error("Avis négatif :
        prioritaire")
else:
    st.success("Avis positif")
```

Les trois tests du TP3 importent `helpers.py` **sans lancer Streamlit**. Une interface non testable est une interface qu'on casse sans s'en apercevoir.

Lancer l'interface, en local et sur Colab

```
streamlit run ui/app.py          # local : le navigateur s'ouvre sur localhost
    :8501
```

Sur Colab, il faut un tunnel

Le serveur tourne sur la machine virtuelle de Colab, que votre navigateur ne peut pas joindre. Le notebook lance `localtunnel`, affiche un mot de passe (l'adresse IP publique de la machine) et une adresse `loca.lt` à ouvrir. Cette adresse est **publique et temporaire** : aucune donnée réelle ne doit y transiter.

```
!streamlit run ui/app.py --server.headless true --server.port 8501 &>/
    content/log &
!npx --yes localtunnel --port 8501
```

Les tests, eux, tournent sans tunnel : ils n'ont besoin que de `helpers.py`.

Plan

L'objectif

Les trois séances

Les ressources partagées

Streamlit

La démarche, pas à pas

CI et MLOps

Les dix-sept contrôles, en un coup d'œil

	Commande	Résultat attendu
T0	`ls` et `ls -a`	7 dossiers, `.gitignore` présent
T1	`pytest -q`	3 failed, 31 passed, 4 skipped
T2	`pytest tests/test_cadrage.py -q`	12 passed
T3	`git status` avant le commit	aucune ligne `.venv/`
T4	`python scripts/check_setup.py`	0 erreur(s)
T5	`git shortlog -sn -all` puis les tests Git	deux noms, 3 passed
T6	diagnostic et `pytest -q`	0 erreur, 0 avertissement, 38 passed
T7	la suite sur un clone frais	38 passed sans intervention
C1	`pytest -q` après le kit TP2	2 failed, 38 passed, 2 skipped
C2	`python src/make_demo.py`	290 avis, 145 de chaque classe
C3	`python src/train.py`	F1 macro 0,879
C4	`python src/evaluate.py`	7 erreurs sur 58, toutes mitigées
C5	`train.py -allocine` puis `pytest -q`	F1 0,920 ; 42 passed ; CI verte
S1	`streamlit run ui/app.py`	la page s'ouvre, trois onglets
S2	un avis mitigé	verdict « à vérifier » sous 60 %
S3	un CSV dans l'onglet 2	métriques, graphique, tableau trié, export
S4	`pytest -q`	45 passed, CI verte

TP1, étapes 1 et 2 : le kit et le venv

```
unzip sentiment-app.zip ; cd sentiment-app
ls ; ls -Force | findstr gitignore          # T0

py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1                # Linux, macOS : source .venv/
    bin/activate
python -m pip install pandas scikit-learn pytest
python -m pytest -q                         # T1
```

T1 doit être ROUGE : 3 failed, 31 passed, 4 skipped

Les 3 échecs sont les fiches vides, les 4 ignorés attendent le dépôt Git. Si vous obtenez 38 passed ici, vous testez un autre dossier.

Un test rouge avant que le travail soit fait est un test utile ; un test vert dès le départ ne vérifie rien.

TP1, étape 3 : les trois fiches de cadrage

Arbre de choix, dans l'ordre : logique connue et stable ? **règles métier**. Données étiquetées ? **machine learning**. Réponse à tirer de documents et à citer ? **RAG**. Sinon seulement, **génératif seul**.

Fiche	Le cas	Réponse attendue
`cas1-pharmacie.md`	Ordonnances incomplètes	Règles métier : (a) pas d'étiquettes, (b) explicabilité, (d) erreur grave
`cas2-reglement.md`	Règlement intérieur	RAG : (b) citer l'article, (a) pas d'étiquettes, (c) volume faible
`cas3-avis-negatifs.md`	Avis négatifs	ML : (a) étiquettes gratuites, (c) volume, (d) erreur peu grave

```
python -m pytest tests\test_cadrage.py -q          # T2 : 12 passed
```

Une seule case - `[x]` ; deux critères cités **par leur lettre** ; chaque rubrique remplie.

TP1, étapes 4 à 8 : le dépôt, les commits, le clone frais

```
git init ; git add -A ; git status          # T3 : aucune ligne .venv/
git commit -m "Semaine 1 : structure du depot et kit TP1"
git branch -M main
git remote add origin https://github.com/votre-compte/sentiment-app.git
git push -u origin main                     # jeton d'accès personnel, pas
    le mot de passe

powershell -ExecutionPolicy Bypass -File scripts\setup.ps1
python scripts\check_setup.py               # T4 : 0 erreur(s)

git pull ; git add README.md ; git commit -m "..." ; git push
git shortlog -sn --all                      # T5 : deux noms distincts
python -m pytest -q                         # T6 : 38 passed, puis CI verte
bash scripts/verif_clone.sh <url>           # T7 : 38 passed sur un clone
    frais
```

T3 est le seul contrôle irréversible : `.venv/` commité, et le test reste rouge même après correction, car il lit tout l'historique.

TP2, étapes A à C : installer, générer, entraîner

```
cd tp2-semaine2 && python appliquer_tp2.py ../sentiment-app
cd ../sentiment-app && python -m pytest -q    # C1 : 2 failed, 38 passed, 2
    skipped

python src/make_demo.py                       # C2 : 290 avis, 145 de chaque
    classe
python src/train.py                           # C3 : F1 macro 0.879
git status --short                            # metrics.json OUI, sentiment.
    joblib NON
```

Les trois lignes de code à lire

Le découpage avant la vectorisation, sinon le jeu de test fuite dans l'entraînement.

Le `Pipeline`, qui enferme vectoriseur et modèle dans un seul objet.

`random_state=42`, sans quoi deux exécutions donnent deux scores.

TP2, étape D : lire les erreurs

```
python src/evaluate.py
```

vérité ↓	nég.	pos.
négatif	**27**	2
positif	5	**24**

7 erreurs sur 58.

Toutes les erreurs sont des avis mitigés

« Service impeccable et livraison en avance sur le papier, **sauf que ensuite** produit cassé à l'arrivée. »

Le modèle compte des mots positifs et négatifs, puis tranche. Il ne voit pas le retournement, parce qu'un sac de mots **n'a pas d'ordre**.

C4 : la limite annoncée en cours devient mesurable. Elle s'écrit dans `docs/semaine2.md`.

TP2, étape E : le vrai jeu

```
python src/train.py --allocine     # 20 000 avis publics, 7,6 Mo, quelques
    secondes
python -m pytest -q                # C5 : 42 passed
```

	Démonstration	Allociné
Avis	290	20 000
Vocabulaire	316 termes	149 529 termes
F1 macro	0,879	**0,920**

Le modèle n'a pas changé : ce sont les données. Et la mesure devient crédible, calculée sur 4 000 avis au lieu de 58, où une erreur de plus déplaçait le score de deux points.

TP3, étapes A à D : l'interface

```
cd tp3-streamlit && python appliquer_tp3.py ../sentiment-app
cd ../sentiment-app && streamlit run ui/app.py        # S1 : localhost:8501
```

S2, l'onglet « un avis »

Trois essais : un positif net, un négatif net, un mitigé. Le mitigé doit afficher **« à vérifier »**.

Faites varier `SEUIL_INCERTITUDE` de 0,50 à 0,75 et observez.

S3, l'onglet « un fichier »

Téléversez `demo_reviews.csv`. Trois métriques, un graphique, un tableau trié négatifs en tête, un export.

L'écart avec le vrai étiquetage, ce sont les erreurs du TP2.

```
python -m pytest -q          # S4 : 45 passed, puis CI verte
```

Le seuil d'incertitude : un arbitrage, pas un réglage

Seuil	Avis envoyés à un agent	Risque
0,50	peu	le modèle tranche seul des cas qu'il ne maîtrise pas
0,60	modéré	compromis retenu par défaut
0,75	beaucoup	la file humaine se remplit, l'automatisation perd son intérêt

Ce curseur ne se règle pas dans le code : il se décide avec l'équipe métier, et il découle directement de la fiche de cadrage du cas 3, qui disait qu'un avis mal classé se requalifie d'un clic. C'est la question que le compte rendu demande d'argumenter, chiffres à l'appui.

Plan

L'objectif

Les trois séances

Les ressources partagées

Streamlit

La démarche, pas à pas

CI et MLOps

Trois artefacts, pas un

Artefact	**Dans Git ?**	**Pourquoi**
Le code (`src/`, `ui/`, `tests/`)	Oui	C'est le travail
L'environnement (`.venv/`)	Non	Reconstructible : `pip install -r`
Les données (`demo_reviews.csv`)	Non	Reconstructible : `make_demo.py`
Le modèle (`sentiment.joblib`)	Non	Reconstructible : `train.py`
La mesure (`metrics.json`)	**Oui**	Elle, ne se reconstruit pas sans réentraîner

Un logiciel classique versionne une chose : le code. Une application intelligente en versionne trois, et ce tableau est la règle qui décide, pour chacune, où elle vit.

Ce que fait l'intégration continue

```yaml
on: [push, pull_request]
  - uses: actions/checkout@v4
    with: { fetch-depth: 0 }          # tout l'historique : sinon un seul
        auteur vu
  - uses: actions/setup-python@v5
    with: { python-version: "3.11" }
  - run: python -m venv .venv && . .venv/bin/activate && pip install pandas
      scikit-learn pytest
  - run: . .venv/bin/activate && python scripts/check_setup.py
  - run: . .venv/bin/activate && pytest -q
```

À chaque push, sur une machine neuve, en Python 3.11 : l'environnement est reconstruit, le diagnostic passe, les tests tournent. Une CI verte veut dire que le dépôt fonctionne **ailleurs que sur vos deux portables**.

Le geste MLOps de la séquence

La CI n'a jamais votre modèle

`models/sentiment.joblib` n'est pas dans le dépôt. Pourtant, l'intégration continue vérifie qu'un modèle acceptable a été produit.

Les 4 tests du modèle	**En CI**
Deux tests lisent `metrics.json`	Ils passent : le fichier est versionné
Deux tests appellent `predict()`	Ils sont ignorés : le modèle est absent

On transporte la **preuve** de la qualité, pas l'artefact lourd. C'est le principe que MLflow et DVC industrialiseront en semaine 7, et la porte de validation qui déclenchera le réentraînement en semaine 9.

DevOps, MLOps : où passe la frontière

DevOps, dans ces trois TP

Environnement reproductible, tests automatisés, versionnage à plusieurs, intégration continue.

Applicable à n'importe quelle application, intelligente ou non.

MLOps, ce qui s'y ajoute

Trois artefacts au lieu d'un. Une validation qui compare une performance à un seuil, au lieu de passer ou d'échouer. Une dégradation possible **sans qu'on ait rien modifié**.

Un test unitaire passe ou échoue. Un modèle a un F1 à comparer à un seuil, sur un jeu de test figé : c'est pour cela que `metrics.json` existe, et que le seuil y est inscrit à côté de la mesure.

Ce qu'il reste à faire, et le compte rendu

La suite du semestre

S3 l'API, S4 la qualité du code, S6 à S8 le paquet, Docker et la CI/CD complète, S7 MLflow et DVC, S9 la dérive et le réentraînement, S10 les agents.

Le compte rendu global

Huit sections, quatre à six pages, dans `docs/compte-rendu.md`, figé par une étiquette Git. Noté sur 20.

Vos chiffres, un exemple d'erreur tiré de vos données, et les échecs réels de vos séances.

Rien de ce que vous avez fait en trois séances ne sera jeté : le dépôt d'aujourd'hui est celui de la semaine 11.